6.2 实战：常用Skills详解

> **生成模型**：OpenAI GPT-5.3 Codex（openai/gpt-5.3-codex）
> **Token 消耗**：输入约 14k，输出约 7k（本节，估算值）

上一节我们把“Skill 是什么”这件事讲清楚了。这一节直接进入实战：把你日常最常用、也最容易提升效率的 6 个 Skills 一次走通。

我们选的对象是：

- `coding-agent`
- `github`
- `weather`
- 浏览器自动化（常见为 `playwright` / `dev-browser`）
- `discord`
- `canvas`

每个 Skill 都按同样结构讲：

1. 它解决什么问题；
2. 怎么安装；
3. 要配什么；
4. 对话里怎么用；
5. 常见坑怎么排查。

你可以把这一节当“随查随用的操作手册”。

6.2.1 coding-agent：把“会聊代码”变成“会落地改代码”

它是干什么的

`coding-agent` 的价值不是“替你变成另一个 IDE”，而是让 Agent 在编程任务里有更稳定的执行策略：

- 先读需求和仓库上下文；

- 再改代码；
- 然后跑测试/构建；
- 最后给你变更说明和验证结论。

通俗讲，它让“代码建议”更容易变成“代码结果”。

安装方式

```
clawhub install coding-agent
openclaw skills info coding-agent
openclaw skills list --eligible
```

依赖与环境

不同团队会搭配不同编程 CLI（例如 codex、claude、opencode、pi 等），你需要按你实际环境补齐。 `skills info` 会提示缺什么。

配置示例

```
{
  skills: {
    entries: {
      "coding-agent": {
        enabled: true,
        config: {
          testFirst: true,
          requireSummary: true
        }
      }
    }
  }
}
```

> **说明**： `config` 字段具体支持哪些键，取决于 Skill 设计。你可以先从最小配置开始，逐步收敛团队约定。

对话使用示例

```
请在当前仓库新增一个 /healthz 接口：
1）返回 { ok: true, ts: <unix ms> }
2）补上单元测试
3）跑测试后告诉我结果
```

再比如：

```
帮我审查这个分支改动，重点看：空指针风险、并发安全、以及是否有破坏兼容性的 API 变更。
```

常见坑

1. **它在“讲道理”，却不真正执行**
    - 多数是工具权限或依赖不足。
2. **代码改了，但没跑验证**
    - 你可以在提示里明确要求“必须跑测试并反馈命令输出摘要”。
3. **修改范围过大**
    - 任务拆小，指定目录和限制条件，稳定性会明显提升。

6.2.2 github：PR/Issue/CI 流程自动化的第一技能

它是干什么的

`github` Skill 让 Agent 按工程化流程去操作 GitHub：

- 拉取 PR 信息；
- 创建/更新 Issue；
- 查询 CI 状态；
- 组织发布说明。

它特别适合“每天重复、但容易出错”的协作动作。

安装方式

```
clawhub install github
openclaw skills info github
```

依赖与认证

核心依赖通常是 `gh` CLI：

```
brew install gh
gh auth login
gh auth status
```

如果你是 CI 环境，可通过令牌方式认证（按你组织规范配置）。

配置示例

```
{
  skills: {
    entries: {
      github: {
        enabled: true,
        env: {
          GITHUB_TOKEN: "$GITHUB_TOKEN"
        },
        config: {
          defaultRepo: "owner/repo",
          defaultBase: "main"
        }
      }
    }
  }
}
```

对话使用示例

```
请帮我看 owner/repo 最近 8 个 PR：
按“可直接合并 / 需要修复 / 风险高”三类输出，并给出理由。
```

根据这个报错日志生成一个 `bug issue`，标题要能让值班同学一眼看懂，正文包含复现步骤和影响范围。

帮我起一个 `PR` 描述：包含 `Summary`、`Testing`、`Risk`、`Rollback Plan` 四段。

常见坑

1. **仓库权限不够**： `gh` 登录账号没有对应 repo 权限。
2. **组织策略限制**：某些组织要求 SSO 或细粒度 token。
3. **仓库上下文不明确**：建议在提示里显式写 `owner/repo` 。

> **衍生解释：为什么要强调“流程化输出”？**
>
> 传统 AI 回答容易“像答案，不像交付物”。GitHub Skill 的核心价值之一，是把回答对齐到团队协作结构（PR 模板、Issue 模板、CI 检查），这类结构化约束是工程效率的关键。

6.2.3 weather：最小依赖、最快见效的 Skill

它是干什么的

`weather` 是一个很经典的“轻技能”：

- 通常不需要复杂账号体系；
- 依赖少（常见只要 `curl` ）；
- 非常适合验证“Skill 注入是否生效”。

安装方式

```
clawhub install weather
openclaw skills info weather
openclaw skills list --eligible
```

配置示例

```
{
  skills: {
    entries: {
      weather: {
        enabled: true,
        config: {
          unit: "metric",
          language: "zh"
        }
      }
    }
  }
}
```

对话使用示例

查一下杭州未来三天天气，按“上班通勤 / 晚上运动 / 周末出行”给建议。

对比北京和深圳本周末天气，告诉我哪个更适合带孩子户外活动。

常见坑

1. **网络问题**：服务源可达性不稳定。
2. **地点歧义**：同名城市可能有多个，最好写“城市+国家/省份”。
3. **时区误解**：跨时区对比时要明确“本地时间”。

6.2.4 浏览器自动化技能：让 Agent 真正“上网操作”

这里我们把和浏览器控制相关的常见 Skill 一起讲，实际名字可能是 `playwright` 、 `dev-browser` ，你的环境里也可能有自定义封装。

它是干什么的

- 打开网页、点击、输入、抓取信息；

- 做回归测试（表单提交流程、登录流程、关键页面可用性）；
- 采集证据（截图、页面状态、错误日志）。

这是从“文本智能体”走向“操作型智能体”的关键一步。

安装方式（示例）

```
clawhub search "playwright"
clawhub install playwright
openclaw skills info playwright
```

如果你的环境用的是另一个技能名，把 `playwright` 换成实际名称即可。

配置示例

```
{
  skills: {
    entries: {
      playwright: {
        enabled: true,
        config: {
          headless: true,
          timeoutMs: 30000,
          screenshotOnError: true
        }
      }
    }
  }
}
```

对话使用示例

```
打开 https://example.com，验证注册流程：
1）填写测试账号
2）提交表单
3）截图关键页面
4）输出失败步骤和复现建议
```

```
帮我抓取这个页面上“价格表”区域，整理成 Markdown 表格。
```

常见坑

1. **验证码/二次验证**：自动化通常卡在登录强验证环节。
2. **页面强依赖前端异步加载**：需要等待策略，不然容易误判。
3. **反自动化机制**：某些站点会拦截自动化流量。

> **衍生解释：为什么浏览器自动化是“生态拐点”？**
>
> 在很多课程里，自动化脚本是“程序员自己写”。而 Skill 化后，普通用户通过自然语言也能驱动自动化流程，这大幅降低了自动化门槛。它不是替代脚本，而是让“需求到脚本能力”的距离变短。

6.2.5 discord：把 Agent 接入社群协作场景

它是干什么的

`discord` Skill 常用于两类任务：

- 运营类：消息整理、公告草稿、活动提醒；
- 协作类：频道巡检、FAQ 自动答复、工单分流。

如果你的团队在 Discord 上协作，这个 Skill 的收益会很直接。

安装方式

```
clawhub install discord
openclaw skills info discord
```

典型配置

```jsonc
{
  skills: {
    entries: {
      discord: {
        enabled: true,
        env: {
          DISCORD_TOKEN: "$DISCORD_TOKEN"
        },
        config: {
          guildId: "123456789012345678",
          defaultChannel: "general",
          maxLinesPerMessage: 20
        }
      }
    }
  }
}
```

对话使用示例

```
请把今天 #release 频道的讨论整理成三段：
1）已确认事项
2）待决策事项
3）风险和责任人
```

```
帮我生成一条发布公告，语气友好，附带升级注意事项和回滚说明。
```

常见坑

1. **机器人权限不足**：看不到频道或无法发消息。
2. **频道上下文太大**：建议给时间范围，避免信息噪音。
3. **换行/长度限制**：不同渠道对消息格式有限制，注意拆分。

6.2.6 canvas：把“文本结论”升级为“可视化交付”

它是干什么的

`canvas` Skill 的核心是可视化输出：

- 生成结构化图文内容；
- 做状态看板、方案草图、流程图式表达；
- 输出可演示、可讨论的成果，而不只是文字描述。

安装方式

```
clawhub install canvas
openclaw skills info canvas
```

配置示例

```
{
  skills: {
    entries: {
      canvas: {
        enabled: true,
        config: {
          theme: "light",
          autoSnapshot: true,
          maxArtifacts: 20
        }
      }
    }
  }
}
```

对话使用示例

把“下季度技术规划”整理成一页可视化画布：
左侧是目标，右侧是里程碑，中间标风险。

基于这个需求文档生成一个登录流程交互草图，标注关键状态与异常分支。

常见坑

1. **输入不够结构化**：只给一句泛描述，输出会发散。
2. **视觉风格预期不一致**：建议在提示里加“风格关键词”。
3. **内容过载**：一页塞太多信息，阅读效果会下降。

6.2.7 六个 Skills 组合拳：一个真实工作日剧本

下面是一个“从早到晚”的组合示例，感受生态协同。

场景：你是项目负责人

1. 早上先用 `github` 查 overnight PR/CI；
2. 用 `coding-agent` 修复高优先级问题并跑测试；
3. 用浏览器自动化技能回归关键页面；
4. 用 `canvas` 输出当天推进看板；
5. 用 `discord` 发团队同步公告；
6. 下班前用 `weather` 安排明天线下活动备选方案。

这不是“炫技流程”，而是把碎片动作串成统一工作流：

- 信息采集 → 代码执行 → 页面验证 → 可视汇报 → 团队同步。

当你这样使用 Skills，就会明显感受到 OpenClaw 的可扩展价值：它不只是会话助手，而是可编排的工作系统。

6.2.8 通用排障清单（所有 Skill 都适用）

当某个 Skill 不符合预期时，按这个顺序查：

```
openclaw skills list
openclaw skills list --eligible
openclaw skills info <skill-name>
openclaw skills check
```

然后逐条确认：

1. **是否被禁用**（`enabled: false`）；
2. **依赖命令是否存在**（如 `gh`、`curl`）；
3. **环境变量是否注入**；
4. **是否被同名 workspace skill 覆盖**；
5. **任务描述是否足够明确触发该场景**。

如果你把这套流程熟练下来，80% 的问题都能快速定位。

6.2.9 学习建议：从“单技能熟练”到“多技能编排”

建议你用三周节奏练：

- **第 1 周**：只练 `github` + `coding-agent`；
- **第 2 周**：加入浏览器自动化，建立“代码改动后自动回归”的习惯；
- **第 3 周**：加入 `canvas` + `discord`，把结果发布和可视化也标准化。

很多同学一开始会追求“装很多 Skill”，但真正拉开差距的，是你能不能把几个关键 Skill 组合成稳定流程。

本节小结

这一节我们把 6 个高频 Skills 的实战路径打通了：

- `coding-agent`：让代码任务从建议走向执行；
- `github`：把 PR/Issue/CI 变成结构化协作流程；
- `weather`：轻量验证技能生效与外部数据调用；
- 浏览器自动化：把 Agent 从“会说”升级到“会上网操作”；
- `discord`：接入团队沟通与信息分发场景；
- `canvas`：把文字输出升级成可视化交付。

你如果已经能稳定用这 6 个 Skill，基本就具备了“生态使用者”的核心能力。下一节我们就进一步升级：不只会用别人的 Skill，而是自己从 0 写一个能跑、能测、能发布的自定义 Skill。